

12 — Contributor Onboarding Brief

Interpretable Machine Learning for Discovering Hidden Regulatory Switches in Non-Coding DNA

Field	Value
Document ID	COB-001
Version	1.0
Status	Active
Created	2026-06-07
Related docs	Charter , Glossary , Technical Design

Welcome

You're joining a computational genomics research project that uses interpretable machine learning to discover candidate regulatory "switch-like" patterns in non-coding DNA. This brief tells you everything you need to know to contribute safely and effectively.

Read time: ~15 minutes

Required reading after this: [Glossary & Project Memory](#) → [Project Charter](#)

1. Project Summary

What we're building

A machine learning pipeline that: 1. Takes annotated non-coding genomic regions from ENCODE (a public database of functional genome elements). 2. Extracts DNA sequences and computes features (k-mer frequencies, motif scores, etc.). 3. Trains interpretable classifiers (logistic regression, random forests, gradient-boosted trees, shallow CNNs). 4. Predicts whether a region is likely regulatory (influencing gene expression) or non-regulatory. 5. Explains which sequence features drive each prediction (feature importance, SHAP, saliency maps). 6. Cross-references important features against known transcription factor binding motifs.

Why it matters

- ~98% of the human genome doesn't code for proteins, but contains regulatory elements that control gene expression.
- Many disease-associated genetic variants are in non-coding regions.
- We need interpretable tools to understand which non-coding sequences have regulatory function and why.
- Large genomic AI models exist but require massive compute and often lack interpretability. We're building the interpretable, accessible alternative.

What it's NOT

- ■ Not a genomic foundation model (we don't train billion-parameter transformers)
- ■ Not a medical diagnostic (no clinical claims)
- ■ Not a wet-lab project (computational only)
- ■ Not a generic AI dashboard (specific scientific task)

- Not a CRISPR editing platform

2. What Is Already Decided

These decisions are **locked**. Don't propose changing them without reading the rationale in [Glossary — Locked Decisions](#).

Decision	Summary
Scope	Interpretable ML for non-coding regulatory DNA. Not a general genome model.
"Regulatory switch"	A working project label for candidate regulatory patterns. Not a claim of molecular proof.
Compute	Kaggle notebooks. No multi-GPU clusters.
No transformer training from scratch	Out of scope due to compute limits.
Starting data	ENCODE cCREs (candidate regulatory elements), K562 cell type primary.
Data split	Chromosome-holdout. Train: chr1–14,X. Val: chr15–18. Test: chr19–22.
Modeling approach	Staged: classical ML → CNN → pretrained embeddings (later).
Interpretability	First-class requirement. Every model must produce explainable outputs.
Region length	1 kb fixed-length regions for initial experiments.

3. What Is NOT Decided Yet

You can contribute to these open questions:

Question	Context
Exact ENCODE experiment accession IDs for K562	Multiple options available; team hasn't finalized
Optimal k-mer range	Will be determined by experiment E3
Whether to include motif scan features in Phase 1	Trade-off between simplicity and feature richness
Which pretrained model for Phase 3	DNABERT-2, Nucleotide Transformer, or HyenaDNA
Publication venue	Workshop, journal, or preprint
Whether to build a demo app	Nice to have, but not core
Code license	MIT, Apache 2.0, or BSD-3

4. Coding Expectations

4.1 Language and Stack

Tool	Version	Notes
Python	≥ 3.10	Primary language for everything
NumPy, pandas, scikit-learn	Latest stable	Core data science stack
XGBoost / LightGBM	Latest stable	Gradient boosting
PyTorch	≥ 2.0	CNN models (Phase 2)
SHAP	≥ 0.42	Feature attribution
matplotlib, seaborn	Latest stable	Plotting
pysam / pyfaidx	Latest stable	Genome sequence access
pybedtools	≥ 0.9	BED file operations

4.2 Code Structure

```
src/  
├── data/           # Data loading, parsing, sequence extraction  
├── features/       # Feature engineering (k-mer, one-hot, GC, motifs)  
├── models/         # Model definitions and registry  
├── interpret/      # Interpretability methods  
├── reports/        # Report generation  
├── tracking/       # Experiment logging  
└── utils/         # Shared utilities
```

Rules: - Write code in `src/` modules, not inline in notebooks. - Notebooks are for exploration, visualization, and Kaggle execution. - Every public function needs a docstring. - Every module needs a brief comment header explaining its purpose.

4.3 Code Style

Rule	Detail
Formatter	Use <code>black</code> with default settings (line length 88)
Lint	Use <code>ruff</code> or <code>flake8</code>
Type hints	Use for all function signatures
Docstrings	Google style
Naming	<code>snake_case</code> for functions/variables; <code>PascalCase</code> for classes
Imports	Grouped: stdlib → third-party → local; one import per line

4.4 Git Workflow

Branch	Purpose
<code>main</code>	Stable, tested code. Direct pushes not allowed.
<code>dev</code>	Active development. Merge to <code>main</code> when stable.
<code>feature/<name></code>	Individual feature branches. Merge to <code>dev</code> via PR.
<code>exp/<name></code>	Experiment-specific branches (optional).

Commit messages: Use conventional commits:

```
feat: add k-mer feature computation for k=3..6
fix: correct off-by-one in chromosome split
data: update ENCODE core download script
exp: add experiment exp_003 XGBoost results
docs: update README with setup instructions
```

5. Research Expectations

5.1 Scientific Integrity

Rule	Why
Never claim causality	We find statistical associations, not causal mechanisms
Report negative results	If a model doesn't work, document why — it's valuable
Acknowledge limitations	Every analysis should mention what it can't tell us
Cite data sources	Always cite ENCODE, JASPAR, reference genome sources
Frame language carefully	Say "candidate regulatory pattern" not "discovered switch"

5.2 Experiment Discipline

Rule	Detail
Name experiments	Use the naming convention: <code>exp_NNN_model_features</code>
Log everything	Config, metrics, data version, software versions, random seed
Use the checklist	Before declaring an experiment complete, run through the reproducibility checklist
Don't skip trivial baselines	Always compare to random and GC-only baselines
Save artifacts	Model, metrics, plots — all go in <code>experiments/</code>

5.3 If You Want to Try Something New

1. Check the Glossary — is it within scope?
2. Check the Modeling Roadmap — is it in the current phase?
3. If yes to both → proceed. Log the experiment.
4. If it's outside current phase → propose at team meeting. Document in "Open Decisions."
5. If it violates a Locked Decision → don't do it without team consensus and documented justification.

6. How to Contribute Safely Without Breaking Scope

6.1 Safe Contributions (Always Welcome)

Contribution type	Examples
Bug fixes	Fix data pipeline errors, correct metric calculations
Documentation	Improve docstrings, add README sections, update glossary
Experiment execution	Run planned experiments from the Modeling Roadmap
Visualization	Create better plots, improve report formatting
Code quality	Add tests, improve modularity, refactor for clarity
Literature	Add papers to annotated bibliography
Feature engineering	New features within the defined feature framework (k-mers, motifs, GC-based)

6.2 Contributions That Need Discussion

Contribution type	Why it needs discussion
New model family	Must fit within current phase and compute constraints
New data source	Must be evaluated for licensing, size, and relevance
Architecture changes	Affects all downstream experiments
Scope expansion	May violate scope lock
Infrastructure changes	Moving off Kaggle, adding databases, etc.

6.3 Contributions That Are Out of Scope

Contribution type	Why it's out of scope
Training a genomic foundation model	Compute limits (Decision D-04)
Adding clinical/diagnostic features	Not a medical product (Decision D-12)

Wet-lab validation workflows	No lab access
Sequences > 10 kb (initially)	Compute limits
Multi-GPU training code	No multi-GPU access

7. Getting Started Checklist

Week 1: Orientation

- ☐ Read this document (you're doing it now ✓)
- ☐ Read [Glossary & Project Memory](#) — understand all terms and locked decisions
- ☐ Read [Project Charter](#) — understand problem, vision, goals, non-goals
- ☐ Skim [Technical Design](#) — understand pipeline structure
- ☐ Set up development environment (see §7.1 below)
- ☐ Run the existing data pipeline end-to-end on a small test subset
- ☐ Review 2–3 papers from the annotated bibliography

Week 2: First Contribution

- ☐ Read [Research Design](#) — understand experiments
- ☐ Read [MLOps Lite](#) — understand logging conventions
- ☐ Pick a task from the project board or task list
- ☐ Create a feature branch, implement, test, submit PR
- ☐ Pair with an existing team member on an experiment run

7.1 Environment Setup

```
# Clone the repository
git clone <repository-url>
cd regulatory-switch-discovery

# Create virtual environment
python -m venv venv
source venv/bin/activate # Linux/Mac
or venv\Scripts\activate # Windows

# Install dependencies
pip install -r requirements.txt

# Verify installation
python -c "import sklearn; import torch; import shap; print('All dependencies OK!)"

# Run tests (if available)
pytest tests/ -v
```

7.2 Kaggle Setup

1. Create a Kaggle account (if you don't have one).
2. Go to Account Settings → Create New API Token → save [kaggle.json](#).
3. Access the team's Kaggle datasets (links will be shared by team lead).
4. Fork the team's Kaggle notebooks to your account for experimentation.
5. When saving results, download and commit to the repository.

8. Communication

Channel	Purpose
Repository Issues	Bug reports, feature requests, experiment proposals
Repository PRs	Code review, experiment results
Team meetings (weekly)	Sync on progress, discuss decisions, plan next steps
Shared doc / Wiki	Meeting notes, literature reviews, design discussions

9. Key Documents Reference

Document	When to read it
01 — Charter	First. Understand what we're building and why.
02 — PRD	When implementing features.
03 — Technical Design	When writing code.
04 — Research Design	When designing experiments.
05 — Dataset Strategy	When working with data.
06 — Modeling Roadmap	When choosing or proposing models.
07 — Compute Memo	When something runs out of memory.
08 — MLOps Lite	When logging experiments.
09 — Risk Register	When something goes wrong.
10 — Roadmap	When planning work.
11 — Glossary	When you encounter an unfamiliar term or want to propose a change.
12 — This document	You're reading it.

10. Final Note

This project is designed to be a serious, long-term research effort, not a weekend hackathon. We value:

- **Correctness** over speed
- **Interpretability** over raw accuracy

- **Reproducibility** over novelty
- **Documentation** over cleverness
- **Honest results** over impressive results

Welcome to the team. ■

End of Document 12 — Contributor Onboarding Brief

Return to: [Document Index](#)